

SMIRK Full System Report

Generated: July 6, 2026
Repository: `/Users/cameronchurch/OpenClaw/workspace/ai-phone-agent-from-gemini`
Branch: `cleanup/stop-tracking-generated-deploy-output`
Live commit verified: `3d6e0dd04076e00e966735f59350c402fc81b4b0`
Primary live app: `https://ai-phone-agent-production-6811.up.railway.app`
Public domain: `https://www.smirkcalls.com`

This report is meant to be readable by a person, but also complete enough for a future AI agent to understand the product, the codebase, the market, the current proof state, and the remaining risks without relying on chat history.

1. Executive Summary

SMIRK is a missed-call recovery SaaS for local service businesses.

The core product loop is:

Caller phones business

-> business misses or cannot answer

-> SMIRK answers inbound

-> AI captures the need and urgency

-> call record, transcript, summary, and contact are stored

-> owner alert and callback task are created

-> dashboard proves the lead was captured and handled

The shortest honest description:

> SMIRK is a Twilio + AI phone agent that catches missed calls for contractors, summarizes what the caller needed, creates callback work, and shows proof in a dashboard.

Current state:

Area	Status
---	---
Live production parity	Green. Live reports `3d6e0dd04076e00e966735f59350c402fc81b4b0`.
First-customer launch gate	Green. `check:first-customer-10of10` passed.
Final-mile 1000/1000 gate	Green. `check:smirk-1000-final-mile` passed with `localScore: 1000`, `productionReady: true`.
Stripe paid handoff proof	Green. Approved signed webhook smoke passed and matched current live deploy.
Smoke cleanup	Green. Approved cleanup deleted the smoke workspace and provisioning request; dry-run now reports zero matches.
Basic/Starter entitlement chaos	Green for current live commit using temporary operator-created Starter workspace; `36` allowed Basic requests passed and `96` Pro-suite requests were blocked.
Webhook reliability	Green for Stage 1. Raw Twilio buffer, guarded replay, lag monitor, and live admin fallback exist.
Market readiness	Strong for first-customer missed-call recovery. Not yet proven across real customer volume.

The product is no longer blocked by stale production. The current blocker is not code; it is real buyer evidence. The next serious proof is a real paid contractor using it on real calls.

2. What Got Done Today

Today's work moved SMIRK from "locally proven but externally bottlenecked" to a verified live 1000/1000 state.

2.1 Production Deploy Parity

Production was moved to:

```
3d6e0dd04076e00e966735f59350c402fc81b4b0
```

Verified by:

```
npm run -s check:live-is-current
```

Result shape:

```
{
  "ok": true,
  "versionHeader": "3d6e0dd04076e00e966735f59350c402fc81b4b0",
  "branchHeader": "cleanup/stop-tracking-generated-deploy-output",
  "appStatus": "ok"
}
```

2.2 Live Admin Webhook Buffer Lag Check

Added a protected live admin route:

```
GET /api/admin/webhook-buffer-lag
```

File:

```
src/routes/admin-maintenance-routes.ts
```

Purpose:

- Let an operator check stale Twilio webhook buffer rows from outside Railway.
- Avoid requiring local access to Railway's private Postgres host.
- Return pending and stale buffer row counts.
- Keep the route protected by dashboard/operator auth.

The external monitor script was updated:

```
scripts/check-webhook-buffer-lag.mjs
```

It now supports two paths:

1. Direct `DATABASE\_URL` query when a database is reachable.
2. Live admin API fallback when local `DATABASE\_URL` is absent or unreachable.

Verified live result:

```
{
  "ok": true,
  "pendingCount": 0,
  "staleCount": 0,
  "code": "WEBHOOK_BUFFER_LAG_OK",
  "source": "live-admin-api",
  "fallbackReason": "missing-database-url",
  "httpStatus": 200
}
```

2.3 Deploy Readiness Guard Fixed For Predeploy Stale State

The deploy readiness flow previously treated the new live auth route as mandatory even before the new route existed in live production. That made the system correctly secure but deploy-blocked.

Updated files:

```
scripts/check-deploy-post-call-fix-ready.mjs
scripts/check-launch-blockers.sh
```

Behavior now:

- Before deploy, operational auth can be marked `blocked-until-deploy` if live is known stale.
- After deploy, `check:ship-live` is strict and must pass the live route.

This is the right split: predeploy should not demand a route that only exists after deploy, but postdeploy must prove the route exists and is protected.

2.4 Full Live Ship Gate Passed

Verified by:

```
npm run -s check:ship-live
```

Coverage included:

- Live app health.
- Live deploy readiness.
- Buyer routes.
- Operational auth.
- Google auth config.

- DB health.
- Domain cutover.
- Proof artifacts.
- Post-call intelligence.
- Dashboard proof.
- Current production fingerprint.
- Latest failed deploy audit.

Important live auth proof:

```
GET /api/admin/webhook-buffer-lag -> 401 without auth
```

That confirms the new admin monitor is not public.

2.5 Stripe Webhook Smoke Passed

Approved command run:

```
ALLOW_AUTO_FULFILL_STRIPE_WEBHOOK_SMOKE=1 npm run check:stripe-webhook-handoff-live
```

Result:

```
{
  "ok": true,
  "webhook": {
    "received": true
  },
  "checkoutStatus": {
    "found": true,
    "request_status": "workspace_created",
    "activation_stage": "setup_required",
    "request_id_exposed": false,
    "stripe_event_id_exposed": false,
    "checkout_session_id_exposed": false,
    "workspace_id_exposed": false
  },
  "cleanupDryRun": {
    "matched_workspaces": 1,
    "matched_provisioning_requests": 1
  }
}
```

Meaning:

- Signed Stripe webhook path works.
- Auto-fulfillment created the expected workspace/provisioning proof.
- Public checkout status did not leak private IDs.
- The smoke record was visible to cleanup.

2.6 Smoke Cleanup Applied

Approved command run:

```
APP_URL=https://www.smirkcalls.com \  
CONFIRM_SMOKE_CLEANUP_APPLY=delete-smirk-smoke-records \  
npm run cleanup:smoke-workspaces:apply
```

Result:

```
{  
  "ok": true,  
  "deleted_workspaces": 1,  
  "deleted_provisioning_requests": 1  
}
```

Final dry-run:

```
{  
  "matched_workspaces": 0,  
  "matched_provisioning_requests": 0  
}
```

2.7 First-Customer Gate Passed

Verified by:

```
npm run -s check:first-customer-10of10
```

Result:

```
{  
  "ok": true,  
  "verdict": "SMIRK first-customer 10/10 gate is fully proven.",  
  "failures": [],  
  "requiredNextApproval": null  
}
```

2.8 Basic Chaos Refreshed Against Current Live Commit

Command shape:

```
APP_URL=https://ai-phone-agent-production-6811.up.railway.app \  
DASHBOARD_API_KEY=<operator-key> \  
ALLOW_SMIRK_BASIC_CHAOS_PROVISION=1 \  
CONFIRM_SMIRK_BASIC_CHAOS_CLEANUP=delete-temp-basic-workspace \  
npm run -s check:basic-chaos
```

Result:

```
{
  "ok": true,
  "gitCommit": "3d6e0dd04076e00e966735f59350c402fc81b4b0",
  "identitySource": "operator-temp-workspace",
  "provisioned": true,
  "cleanedUp": true,
  "allowedRequests": 36,
  "restrictedRequests": 96,
  "code": "BASIC_CHAOS_PASSED",
  "cleanupRequired": false
}
```

Meaning:

- A temporary Starter workspace was created.
- Basic-allowed routes stayed accessible.
- Pro-suite routes returned `PRO\_SUITE\_REQUIRED`.
- No Pro-suite metrics leaked into Basic responses.
- The temporary workspace was deleted.

2.9 Final-Mile 1000/1000 Gate Passed

Verified by:

```
npm run -s check:smirk-1000-final-mile
```

Result:

```
{
  "ok": true,
  "localScore": 1000,
  "targetScore": 1000,
  "localFinalMileComplete": true,
  "productionReady": true,
  "failures": []
}
```

3. What The App Is

SMIRK has three layers:

1. A public buyer surface.
2. A customer dashboard.
3. An operator/admin cockpit.

3.1 Public Buyer Surface

Purpose:

- Explain missed-call recovery.
- Show pricing.
- Accept setup requests.
- Start Stripe checkout.
- Show checkout success/cancel flows.
- Provide public proof counters without leaking sensitive data.

Representative routes:

```
GET /
GET /book
GET /success
GET /cancel
GET /api/pricing
GET /api/first-dollar-readiness
GET /api/public-proof-snapshot
POST /api/checkout/create
POST /api/provisioning/request
POST /api/provisioning/checkout-status
```

3.2 Customer Dashboard

Purpose:

- Help owners and staff answer five questions:

1. Who called?
2. What did they need?
3. How urgent was it?
4. Who needs to call them back?
5. Did it get handled?

Basic/Starter dashboard:

```
Calls
Contacts
Tasks
```

Pro/Agency dashboard:

```
Calls
Contacts
Tasks
CRM-style views
Appointments
Handoffs
Recovery queue
Analytics
Additional customer tools
```

Operator-only surfaces must not appear to customers.

3.3 Operator/Admin Cockpit

Purpose:

- Workspaces.
- Provisioning requests.
- Logs.
- Health.
- System settings.
- Compliance/DNC.
- Voice and agent config.
- Proof and deploy controls.
- Smoke cleanup.
- Webhook lag/replay.

This is the machine room. It is valuable for the operator, but it is not the product a contractor should live inside.

4. Core Architecture

4.1 Runtime Stack

| Layer | Current implementation |

| --- | --- |

| Frontend | React + Vite + TypeScript in `src/App.tsx` and `src/main.tsx`. |

| Backend | Express server in `server.ts`. |

| Database | Postgres via `postgres.js` in `src/db.ts`. |

| Telephony | Twilio webhook routes and TwiML handling. |

| AI brain | OpenClaw/OpenRouter/Gemini-style configured paths with tool calling. |

| TTS | Multiple TTS providers exist: ElevenLabs, Cartesia, OpenAI, Google TTS. |

| Billing | Stripe checkout and signed webhook handling. |

| Email alerts | Resend-based owner email path. |

| Deployment | Railway. |

| Auth | Dashboard API key, workspace bearer tokens, Google OAuth, provisioning secret, Twilio validation where configured. |

| Guard scripts | Extensive `scripts/check-\*` gates and deploy wrappers. |

4.2 Call Flow

```
Twilio voice webhook
-> Express route
-> webhook_event_buffer captures raw payload
-> call/contact/message rows
-> AI conversation loop
-> tool dispatch
-> transcript
-> post-call intelligence
-> summary/outcome/task/event
```

```
-> owner alert
-> dashboard proof
```

4.3 Data Model

Main data concepts:

| Concept | Purpose |

| --- | --- |

| `workspaces` | Customer/business accounts with plan, owner, API key, limits, and business identity. |

| `contacts` | Caller identity, DNC flag, notes, tags, status, last summary/outcome. |

| `calls` | Twilio call SID, direction, status, contact/business/workspace linkage, recording metadata. |

| `messages` | Per-turn transcript rows. |

| `call\_summaries` | Post-call summary and outcome data. |

| `call\_events` | Timeline and proof events. |

| `tasks` | Callback/follow-up work. |

| `appointments` | Scheduling data where enabled. |

| `handoffs` | Human escalation records. |

| `tool\_executions` | Audit trail for AI tool calls. |

| `request\_logs` | Operational request log. |

| `webhook\_event\_buffer` | Raw Twilio intake buffer for durable call recovery. |

| `provisioning\_requests` | Buyer/Stripe onboarding records. |

| `activation\_events` | Workspace activation lifecycle. |

The production architecture still uses shared Postgres with workspace-level isolation. Schema-per-tenant and distributed database work are roadmapped, not implemented. That is intentional because the current routes depend on the existing schema and because real usage has not justified a high-risk database rewrite.

4.4 Route Inventory

Evidence:

```
openapi.yaml matches 197 concrete API route declarations
rough route registration grep count: 201
```

The slight difference is expected because raw grep counts helper or non-OpenAPI route registrations differently. The canonical API route contract is the OpenAPI check:

```
npm run -s check:openapi
```

4.5 No-DB Demo Mode

No-DB mode allows the app to boot without `DATABASE\_URL`.

Purpose:

- Local demos.

- Sales walkthroughs.
- Zero external billing footprint.
- Fast laptop proof without Postgres setup.

Verified by:

```
npm run -s check:no-db-demo-mode
```

Current evidence:

```
{
  "workspacePlan": "starter",
  "calls": 3,
  "contacts": 3,
  "tasks": 3,
  "reviewItems": 2,
  "code": "NO_DB_DEMO_MODE_PASSED"
}
```

Important distinction:

- No-DB mode uses curated demo data by design.
- Live readiness and first-customer proof do not depend on No-DB mock data.

5. Product Boundaries And Plans

5.1 Current Pricing Shape

From `src/saas.ts`:

Plan	Internal id	Current label	Limits
Free	`free`	Free Trial	50 calls, 100 minutes, 1 agent
Starter	`starter`	Starter - \$197/mo	500 calls, 1000 minutes, 3 agents
Pro	`pro`	Pro - \$397/mo	2000 calls, 5000 minutes, 9 agents
Enterprise/Agency	`enterprise`	Agency - \$697/mo	unlimited calls/minutes/agents

5.2 Entitlement Rule

Starter/Basic users should get the basic dashboard.

Pro/Agency users should get the full customer suite.

Operator/admin users get the machine room.

This is not only a UX preference. It is the pricing model.

If Basic can see Pro dashboards or Pro metrics, pricing collapses. If Pro is forced into the tiny Basic dashboard, Pro has no reason to exist.

Verified by:

```
npm run -s check:customer-dashboard  
npm run -s check:plan-boundaries  
npm run -s check:basic-chaos
```

5.3 Customer Promise By Plan

Basic/Starter:

We catch missed calls and give you Calls, Contacts, and Tasks.

Pro:

We give you the full operating suite around recovered calls: deeper dashboard, recovery, handoffs,

Operator/admin:

We run and prove the platform.

6. Security, Compliance, And Guardrails

6.1 Auth Boundaries

Protected surfaces include:

- Operator session.
- Calls.
- Tasks.
- Contacts.
- Settings.
- Logs.
- Agent config.
- Compliance/DNC.
- Integrations.
- System health diagnostics.
- Workspace provisioning.
- Admin maintenance.
- Webhook buffer lag.
- Prospecting.
- Chat/debug surfaces.

Live auth audit confirmed operational endpoints reject unauthenticated public access while buyer validation routes remain reachable.

6.2 Production CORS

Production CORS defaults are hardened to known SMIRK origins and preserve authenticated browser headers.

Verified by:

```
npm run -s check:cors-security
```

6.3 DNC And Contact Controls

Current DNC/contact behavior includes:

- Contact status editing.
- Do-not-call flag.
- DNC consent/correction notes.
- Compliance audit route.
- DNC delete/removal path.
- Contract check for contact management.

Verified by:

```
npm run -s check:contact-management
```

Important legal boundary:

SMIRK has compliance guardrails. It is not legal advice. SMS, outbound calling, DNC release flows, and consent language still need policy review before being used as aggressive outbound automation.

6.4 Approval-Gated Production Mutations

The repo intentionally does not let dangerous production writes happen silently.

Examples:

| Mutation | Guard |

| --- | --- |

| Production deploy | `CONFIRM\_SMIRK\_POST\_CALL\_FIX\_DEPLOY=deploy-post-call-fix` plus branch confirmation. |

| Stripe webhook smoke | `ALLOW\_AUTO\_FULFILL\_STRIPE\_WEBHOOK\_SMOKE=1`. |

| Smoke cleanup apply | `CONFIRM\_SMOKE\_CLEANUP\_APPLY=delete-smirk-smoke-records`. |

| Basic chaos temp workspace cleanup | `CONFIRM\_SMIRK\_BASIC\_CHAOS\_CLEANUP=delete-temp-basic-workspace`. |

| Webhook replay apply | Separate replay apply command. |

| Real proof call | Readiness and target safety gates. |

This is one of SMIRK's strongest engineering traits: production proof exists, but high-risk writes require explicit confirmation.

7. Reliability And Operations

7.1 Current Reliability Spine

Implemented:

- Raw Twilio webhook event buffer.
- Guarded replay worker.
- Webhook lag checker.
- Live admin fallback for lag checking.
- Deploy fingerprint stamping/checking.
- Live parity gate.
- Failed deploy gate.
- Domain cutover checks.
- DB health checks.
- Public proof freshness checks.
- Smoke cleanup.

7.2 Webhook Buffer

Table:

```
webhook_event_buffer
```

Purpose:

- Capture raw Twilio payloads before heavier processing.
- Preserve evidence if post-processing fails.
- Enable replay and stale-row monitoring.

Current check:

```
WEBHOOK_BUFFER_LAG_MAX_AGE_MINUTES=5 npm run -s check:webhook-buffer-lag
```

Live result:

```
0 pending, 0 stale
```

7.3 What Is Not Implemented Yet

Not implemented:

- Redis call-session cache.
- Schema-per-tenant database isolation.
- Distributed multi-region database grid.
- Automatic cross-region failover.
- Long-term volume-based database partitioning.

These are roadmap items. They should be driven by real traffic and revenue, not by architecture fantasy.

8. Verification State

Current proof commands that matter:

```
npm run -s check:live-is-current
npm run -s check:latest-failed-deploy
npm run -s check:ship-live
WEBHOOK_BUFFER_LAG_MAX_AGE_MINUTES=5 npm run -s check:webhook-buffer-lag
npm run -s check:first-customer-10of10
npm run -s check:smirk-1000-final-mile
APP_URL=https://www.smirkcalls.com npm run -s cleanup:smoke-workspaces
```

Current verified results:

```
| Gate | Result |
| --- | --- |
| `check:live-is-current` | Passed. |
| `check:latest-failed-deploy` | Passed. No failed deployments found. |
| `check:ship-live` | Passed. |
| `check:webhook-buffer-lag` | Passed. `0` pending, `0` stale. |
| `check:first-customer-10of10` | Passed. |
| `check:smirk-1000-final-mile` | Passed. |
| `cleanup:smoke-workspaces` dry-run | Passed. `0` smoke records. |
| `git status --short` | Clean at last verification. |
```

9. Market Comparison

This section uses current open-market sources checked July 2026. The market is active and pricing/features can change, so rerun research before using this in a pitch deck.

9.1 Market Categories

Category A: Missed-call text-back and lightweight AI answering

Examples:

- Upfirst.
- Allo/Rosie-style products.
- GoHighLevel-style automations.
- Other text-back workflow vendors.

Market behavior:

- These products emphasize low-cost phone coverage.
- Many include missed-call texts, summaries, basic routing, and appointment links.
- Pricing can start low, but call count, booking, transfer, and feature gates matter.

Open-market evidence:

- Upfirst positions itself as an AI answering service for small businesses and says it can take messages, answer questions, book appointments, transfer calls, and send summaries by text/email. Source: <https://upfirst.ai/>
- Upfirst's missed-call text-back page says it answers calls, sends follow-up texts, captures lead details, books appointments, screens calls, and transfers hot leads. Source: <https://upfirst.ai/solutions/missed-call-text-back>
- Upfirst's pricing comparison says Upfirst starts at \$24.95/month for 30 calls, with a 14-day no-credit-card trial. Source: <https://upfirst.ai/blog/most-affordable-ai-answering-service-comparison>

- Allo/Rosie pricing is reported as \$49/month for Professional, \$149/month for Scale, and \$299/month for Growth in an Allo article. Source: <https://www.withallo.com/blog/top-ai-medical-answering-services>

SMIRK implication:

SMIRK should not try to win on cheapest sticker price. It should win on contractor-specific call capture, owner proof, callback accountability, and plan-gated operational clarity.

Category B: Broad AI receptionist platforms

Examples:

- RingCentral AI Receptionist.
- Aircall AI Voice Agent.
- Smith.ai.
- CloudTalk/CeTe-style AI scheduling tools.

Market behavior:

- Broader than missed-call recovery.
- Often includes FAQ handling, routing, appointment scheduling, texts, integrations, analytics, and enterprise phone-system workflows.
- Useful for offices, contact centers, law firms, clinics, and businesses with more structured operations.

Open-market evidence:

- RingCentral AI Receptionist markets missed-call capture, FAQs, texts, appointment scheduling, lead capture, routing with context, multilingual/human-like interactions, and setup in minutes. Source: <https://www.ringcentral.com/pricing/ai-receptionist.html>
- Aircall describes AI Voice Agent as answering inbound calls, handling FAQs, capturing details, transferring with context, supporting multiple languages, and integrating into operations. Source: <https://aircall.io/blog/features/ai-voice-agent/>
- Smith.ai markets real-time system connections, dynamic pricing/status lookups, conditional routing, qualification workflows, and automation agents. Source: <https://smith.ai/>

SMIRK implication:

SMIRK should not pitch itself as a universal receptionist. That battlefield is crowded and feature-heavy. SMIRK's sharper wedge is:

Missed call -> useful callback work -> owner proof

Category C: Booking-first AI agents

Example:

- Voksha.

Open-market evidence:

- Voksha positions around turning missed calls into booked appointments, including booking/rescheduling/canceling appointments, answering service/pricing/hours questions, sending confirmations/reminders, and capturing new-client details. Source: <https://voksha.com/>

SMIRK implication:

SMIRK currently has calendar/appointment code, but its first-dollar product should avoid promising full autonomous booking. For contractors, the safer first promise is callback-ready qualification, not unsupervised scheduling.

9.2 Pricing Reality

The market has cheap entry-level AI answering services. SMIRK's current `starter` price of `\$197/mo` is not a bargain-bin price.

That means the pitch must justify the price through:

- Contractor-specific urgency.
- High-ticket missed-call economics.
- Dashboard proof.
- Owner alerts.
- Callback accountability.
- Compliance-conservative inbound posture.
- Higher trust and implementation support.

If SMIRK tries to sell "generic AI receptionist" at \$197/month against \$25-\$49/month entry offers, it will struggle.

If SMIRK sells "recover one plumbing/HVAC/roofing job you otherwise lost," \$197/month is more defensible.

9.3 Where SMIRK Wins

SMIRK wins when:

- The buyer is phone-driven.
- A missed call has real job value.
- The owner is in the field and cannot always answer.
- Speed-to-lead matters.
- A callback task is more realistic than autonomous booking.
- The buyer wants proof of recovered work.
- The product hides the machine room.

Best early niches:

```
Plumbing
HVAC
Roofing
Electrical
Handyman
Water damage / restoration
Auto repair
Landscaping with high estimate volume
```

9.4 Where SMIRK Loses

SMIRK loses when:

- The buyer wants the cheapest AI receptionist.
- The buyer needs full calendar booking as the main feature.
- The buyer wants SMS-first automation.
- The business does not depend on phone leads.
- The owner expects a polished, mass-market self-serve onboarding flow with no operator involvement.
- The UI exposes too many operator tools.

9.5 Market Verdict

SMIRK is not alone. The market is very active.

The winning strategy is not to add more generic AI receptionist features. The winning strategy is to make the narrow contractor missed-call loop extremely easy to understand, buy, activate, and prove.

10. Buyer-Specific Understanding

10.1 Homeowners

Homeowners do not buy SMIRK.

They experience it when:

- Their call is answered.
- Their problem is understood.
- They do not feel ignored.
- The contractor calls back with context.

SMIRK should be invisible to the homeowner. It should feel like the business is responsive.

10.2 Handymen

Handymen care about:

- Missed jobs.
- Being on a ladder, in a crawlspace, driving, or with another customer.
- Avoiding another dashboard.
- Knowing who to call back first.

Best pitch:

When you miss the call, SMIRK catches the job and tells you who needs a callback.

10.3 SMB Owners

SMB owners care about:

- Revenue leakage.
- Staff overload.
- Lead quality.
- Proof.
- Simplicity.

Best pitch:

Your phone is a revenue channel. SMIRK keeps missed calls from becoming lost jobs.

10.4 Office Managers / Dispatchers

They care about:

- Clean queues.
- Callback ownership.
- Urgency.
- Contact history.
- Completed work.

They need:

```
Calls
Contacts
Tasks
Status
```

They do not need:

```
deploy logs
AI model config
Twilio internals
operator health panels
raw telemetry
```

10.5 Developers

Developers should treat scripts and live artifacts as the source of truth.

Important commands:

```
npm run build
npm run lint
npm audit --audit-level=moderate
npm run -s check:openapi
npm run -s check:auth-regression
npm run -s check:customer-dashboard
npm run -s check:plan-boundaries
npm run -s check:smirk-1000-final-mile
```

Important rule:

If prose says one thing and a gate says another, the gate wins.

10.6 AI Agents

Future AI agents should start with:

```
git status --short
git rev-parse HEAD
npm run -s check:live-is-current
```

```
npm run -s check:smirk-1000-final-mile
npm run -s check:first-customer-10of10
```

Do not claim readiness from README text alone. The README can drift.

11. What Is Broken, Weak, Or Not Proven

11.1 Real-Customer Repetition Is Missing

The system is verified through gates and smoke tests. That is strong engineering proof.

It is not the same as:

- Ten real contractors onboarding.
- Hundreds of real customer calls.
- Real cancellation/churn data.
- Real confused-user feedback.
- Real setup friction data.

This is the biggest remaining product risk.

11.2 Customer UI Still Needs Continued Discipline

The customer/operator partition is now contract-tested.

Risk remains:

- New features can leak operator concepts into customer navigation.
- Pro can become too broad.
- Basic can become too weak.
- The product can slide back into an "internal cockpit with a public door."

Control:

```
npm run -s check:customer-dashboard
npm run -s check:plan-boundaries
npm run -s check:basic-chaos
```

11.3 Documentation Drift Exists

The README contains useful source-of-truth commands, but some status prose can become stale after a deploy. The current verified state is better represented by:

```
output/smirk-1000-final-mile-audit.json
output/first-customer-10of10-readiness.json
output/basic-chaos-last.json
output/webhook-buffer-lag.json
```

Control:

- Treat reports as timestamped snapshots.

- Treat commands as live truth.
- Update README only after gate state changes.

11.4 Enterprise Database Architecture Is Roadmapped

The "sovereign multi-tenant database grid" is not implemented.

That is correct for now.

The current production state is:

- Shared Postgres.
- Workspace-scoped rows.
- Durable webhook buffer.
- Replay and lag monitoring.

The future path is:

1. More monitoring.
2. Real traffic evidence.
3. Redis only if latency proves it is needed.
4. Export/restore tooling.
5. Schema-per-tenant for high-value enterprise only after revenue justifies it.

11.5 SMS Is Not The First-Dollar Product

There is legacy SMS-related storage and market pressure around missed-call text-back.

But the current first-dollar product should not be sold as SMS automation.

Control:

```
npm run -s check:no-texting-copy  
npm run -s check:first-dollar-offer-scope
```

12. Recommended Go-To-Market

12.1 Do Not Sell "AI Receptionist"

That is crowded, generic, and price-compressed.

Sell:

```
Missed-call recovery for local service businesses.
```

12.2 Best Offer

```
We catch calls you miss, summarize what the customer needed, alert you, and create the callback ta
```

12.3 First Niche

Pick one:

Plumbing
HVAC
Roofing
Electrical
Handyman

Do not sell all industries at once.

12.4 First Proof Asset

Show a real or No-DB demo dashboard with:

- One emergency call.
- One contact.
- One transcript.
- One callback task.
- One proof counter.

Do not demo the operator cockpit.

12.5 Pricing Guidance

Current pricing can work if the buyer believes one recovered job pays for the month.

If selling to very small handymen:

- Starter at \$197/month may feel high unless the demo clearly shows recovered job value.
- A lower entry plan or setup-assisted pilot may reduce friction.

If selling to plumbing/HVAC/roofing:

- \$197/month is more defensible.
- Pro at \$397/month needs a clear reason: multi-user, higher volume, deeper recovery/handoff/analytics.

13. AI Agent Handoff Packet

13.1 Start Here

```
cd /Users/cameronchurch/OpenClaw/workspace/ai-phone-agent-from-gemini
git status --short
git rev-parse HEAD
git branch --show-current
```

13.2 Verify Current Truth

```
npm run -s check:live-is-current
npm run -s check:latest-failed-deploy
npm run -s check:smirk-1000-final-mile
npm run -s check:first-customer-10of10
APP_URL=https://www.smirkcalls.com npm run -s cleanup:smoke-workspaces
```

13.3 Important Files

| File | Why it matters |

| --- | --- |

| `README.md` | Human overview and command list. Can drift. |

| `server.ts` | Express app and core runtime wiring. |

| `src/App.tsx` | Frontend dashboard and public site. |

| `src/db.ts` | Core Postgres schema. |

| `src/saas.ts` | Workspace, billing, plans, provisioning. |

| `src/function-calling.ts` | AI tool declarations and dispatch. |

| `src/intelligence.ts` | Post-call summary/task/proof logic. |

| `src/compliance.ts` | DNC/compliance helpers. |

| `src/routes/admin-maintenance-routes.ts` | Admin maintenance, smoke cleanup, webhook lag route. |

| `scripts/check-smirk-1000-final-mile.mjs` | Final-mile combined audit. |

| `scripts/check-first-customer-10of10-readiness.mjs` | Launch-readiness gate. |

| `scripts/check-basic-chaos.ts` | Starter/Basic entitlement stress test. |

| `scripts/check-webhook-buffer-lag.mjs` | Live webhook lag monitor. |

| `deploy.sh` | Guarded deploy wrapper. |

| `docs/SMIRK\_1000\_ROADMAP.md` | Roadmap and database architecture boundary. |

13.4 Do Not Do These Without Explicit Approval

- Production deploy.
- Stripe webhook smoke.
- Smoke cleanup apply.
- Real proof call.
- Webhook replay apply.
- Outbound calling or SMS campaigns.
- Secret rotation.
- Database destructive actions.

13.5 Current Best Verdict

SMIRK is live-current, first-customer-gate proven, final-mile 1000/1000 proven, and ready for a co

The next milestone is not another architecture sprint. It is real buyer onboarding and real call evidence.

14. Source Links For Market Section

- RingCentral AI Receptionist pricing/features: <https://www.ringcentral.com/pricing/ai-receptionist.html>
- Upfirst homepage: <https://upfirst.ai/>
- Upfirst missed-call text-back page: <https://upfirst.ai/solutions/missed-call-text-back>
- Upfirst pricing comparison: <https://upfirst.ai/blog/most-affordable-ai-answering-service-comparison>
- Allo/Rosie pricing article: <https://www.withallo.com/blog/top-ai-medical-answering-services>
- Voksha homepage: <https://voksha.com/>
- Smith.ai homepage: <https://smith.ai/>
- Aircall AI Voice Agent article: <https://aircall.io/blog/features/ai-voice-agent/>

15. Bottom Line

SMIRK is technically much stronger than a normal side-project MVP. It has real production gates, live parity, Stripe proof, cleanup proof, plan boundaries, Basic chaos validation, guarded deploys, public proof counters, and a durable webhook reliability spine.

The business risk is not "can the code run?" anymore.

The business risk is:

Can a non-technical contractor understand it, trust it, pay for it, and keep using it after the fi

That is the next battlefield.